Book 5

Every Action, Explained

The complete reference — all 122 actions

Every action Genius can take, with the exact name it is called by, what it does in plain English, whether Ableton has to be running, how expensive it is, and every argument it accepts. The names, descriptions and arguments in this book are extracted directly from the shipping source at build time, so they cannot drift out of date the way a hand-written list does.

In this book

- All 122 actions in 17 families
- A plain-English explanation of each, not just its machine description
- Cost weighting, so you know what's cheap and what's expensive
- Whether Ableton needs to be running — more actions work offline than people expect
- Every argument, which are required, and what each one means
- Inert and unreliable actions flagged in place

Extracted from app/genius_client.py at build time. Behaviour verified against Ableton Live 12.4.15b3.

How to read an entry

Field	Meaning
Weight	Roughly what the call costs to run. **Feather** — one tiny bridge round-trip, effectively instant; **Light** — one round-trip, small payload back; **Medium** — one big payload, or a handful of chained calls; **Heavy** — loops over many tracks/clips/notes, or heavy local compute; **Very heavy** — real-time wall-clock work, or a full-session scan.
Needs Live	**Yes** — requires Ableton running with the bridge device loaded. **No** — runs entirely locally; Ableton can be closed. **Hybrid** — computes locally, then writes the result into Live.
In plain terms	What it actually does and why you'd want it — written for a musician, not a developer.
Machine description	The exact text Genius itself sees when deciding whether to call this action. Extracted verbatim from the source.
Arguments	Every parameter, required ones first. You rarely need to supply these yourself — Genius fills them in — but they tell you exactly what the action can be asked to vary.
Note	Practical guidance: gotchas, costs, whether it's destructive, and any known problems.

Weight scale used throughout:

FEATHER LIGHT MEDIUM HEAVY VERY HEAVY

Flagged actions

Five actions in this book carry a warning in their note: four are confirmed inert or unreliable, and one is shipping but under-tested. They are documented rather than hidden because they are still in the tool list and Genius may still call them. Book 4 has the full evidence.

The families

Family	n	Actions
Connect & Read Live State	7	`live_connect, live_read_set, live_get_selected, live_session_snapshot, live_session_diff, live_read_link_status, live_read_meter`
Transport, Tempo & Global Feel	22	`live_update_set, live_playback, live_undo, live_redo, live_jump_to_time, live_set_overdub, live_tap_tempo, live_continue_playing, live_scrub, live_jump_cue, live_set_punch, live_set_cue_volume, live_stop_all_clips, live_batch_tracks, live_set_crossfader, live_set_loop_region, live_set_groove_amount, live_nudge, live_set_swing, live_set_automation_record, live_back_to_arranger, live_re_enable_automation`
Tracks & Mixer	8	`live_read_track, live_update_track, live_create_track, live_edit_track, live_delete, live_duplicate, live_select, live_set_fold_state`
Scenes	4	`live_read_scene, live_update_scene, live_create_scene, live_edit_scene`
Session Clips — Create & Edit	15	`live_read_clip, live_update_clip, live_create_clip, live_edit_clip, live_write_notes, live_remove_notes, live_duplicate_loop, live_commit_groove, live_groove_pool, live_set_clip_fades, live_set_clip_markers, live_manage_warp_markers, live_set_clip_follow_action, live_get_clip_follow_action, live_set_clip_launch_mode`
Clip Automation & Envelopes	4	`live_set_clip_automation, live_get_clip_automation, live_add_shaped_automation, live_clear_clip_envelopes`
Arrangement Timeline	10	`live_record_to_arrangement, live_duplicate_to_arrangement, live_read_arrangement_clips, live_update_arrangement_clip, live_move_arrangement_clip, live_read_arrangement_notes, live_write_arrangement_notes, live_remove_arrangement_notes, live_read_arrangement_length, live_capture_midi`
Take Lanes & Comping	4	`live_create_take_lane, live_read_take_lanes, live_update_take_lane, live_create_take_lane_clip`
Devices, Racks & Macros	13	`live_read_device, live_update_device, live_create_device, live_move_device, live_read_macros, live_update_macro, live_map_rack_magnitude, live_read_drum_pads, live_update_drum_pad, live_inspect_plugin, live_read_param_range, live_set_param_raw, live_set_appointed_device`
Algorithmic & Generative MIDI	8	`live_generate_euclidean, live_generate_chords, live_generate_walking_bass, live_transform_clip, live_write_drum_grid, live_create_drum_pattern, live_create_chord_progression, live_detect_key`
MIDI Editor Note Selection	2	`live_read_selected_notes, live_transform_selected_notes`
Genre Recipes	2	`live_list_recipes, live_apply_recipe`
Audio Analysis, Recording & Mix	5	`live_analyze_audio, live_compare_audio, live_record_audio, live_record_and_analyze, live_analyze_mix`
Samples & Browser	2	`live_library, live_find_samples`
Knowledge Base	3	`library_db_search, library_search, library_read`
Offline Project Files (.als)	5	`als_summary, als_list_tracks, als_extract_midi, als_diff, als_find_unfinished`
Diagnostics, Views & Escape Hatches	8	`live_context, live_api, live_run_action, live_focus_view, live_view_control, live_show_message, live_read_dialog, live_read_cpu`

Connect & Read Live State

Start here. Nothing else works until the bridge answers, and almost every good multi-step prompt begins by reading the room before touching anything.

`live_connect`

FEATHER LIVE: YES

The handshake. Pings Ableton through the Live Actions Bridge and comes back with the Set's name, tempo, scale and how many tracks and scenes exist. If this fails, every other live_* action will fail too, so it's the first thing to try when something seems broken.

Machine description: *Connect / ping Ableton through Live Actions Bridge. Call first when the user wants Live work. Returns set name, tempo, scale, track/scene counts.*

Arguments: none — just call it.

> **Note:** Genius is told to call this first whenever you ask for Live work. Safe to spam; changes nothing.

`live_read_set`

MEDIUM LIVE: YES

The wide-angle photo of your Set. By default you get tracks and scenes; ask for more and it will also return routings, mixer values, track colours and Arrangement locators. This is how Genius learns what your project actually contains instead of guessing.

Machine description: *Read Live Set globals plus optional detail. Default include is tracks+scenes.*

Argument	Type		What it's for
`include`	array: tracks | scenes | routings | mixer | color | locators | *	opt	tracks, scenes, routings, mixer, color, locators, or * for all.

> **Note:** A 9-track Set returns roughly 16 KB in about 0.13s. Asking for '*' on a large Set is the single biggest payload in the toolset — request only the sections you need.

`live_get_selected`

FEATHER LIVE: YES

Answers 'what am I pointing at right now?' — the track, scene, clip and device currently highlighted in Live's own GUI. Lets you say 'this clip' instead of counting track numbers.

Machine description: *Inspect what the user currently has selected in Ableton Live's GUI (selected track index/name/type, selected scene index/name, selected device index/name/class, and detail clip). Call this first when the user asks 'what is this?' or refers to their current selection.*

Arguments: none — just call it.

> **Note:** The cheapest way to give Genius context. Pair it with any edit action to mean 'the thing I'm looking at'.

`live_session_snapshot`

HEAVY LIVE: YES

Takes a deep JSON photograph of the whole session and hands it back to you. On its own it does nothing useful — its job is to be the 'before' half of a before/after comparison.

Machine description: *Read-only deep structural snapshot of the Live session (tempo, signature, playing state, song time, tracks with volume/pan/mute/solo/arm, clip slots, and devices). Use before and after mutating tools to verify changes with live_session_diff.*

Argument	Type		What it's for
`include_clips`	boolean	opt	Include session clip slots and playing states (default true).
`include_devices`	boolean	opt	Include device names and class names (default true).

Note: Walks the entire Set. Take one before a risky batch of edits so you can prove exactly what changed.

live_session_diff

MEDIUM LIVE: HYBRID

Feed it an earlier snapshot and it tells you precisely what is different now — which tracks, clips, devices and values moved. This is the receipt for a big automated change.

Machine description: *Compare a previous session_snapshot with the current Live state to verify what actually changed. Returns a detailed list of added/removed/changed paths, before/after values, and total change count.*

Argument	Type		What it's for
previous	object	req	The previous snapshot object returned by live_session_snapshot.

Note: The comparison itself runs locally in Python; only the fresh 'after' read touches Live.

live_read_link_status

FEATHER LIVE: YES

Reports Ableton Link and external sync state: how many peers are connected, the quantum, and whether tempo is being driven from outside.

Machine description: *Read-only Ableton Link and external sync status discovery (is_link_enabled, num_peers, is_connected, quantum, start_stop_sync_enabled, tempo_sync_enabled).*

Arguments: none — just call it.

Note: Useful when the tempo won't respond to live_update_set — Link may be overriding it.

live_read_meter

FEATHER LIVE: YES

Reads the live peak/VU output level on a track, a return, or the master, the same numbers the meters in Live are showing right now.

Machine description: *Read real-time peak/VU output meters (output_meter_left, output_meter_right, output_meter_level 0.0-1.0) from a track, return track, or master bus while playback is running.*

Argument	Type		What it's for
track_number	integer	opt	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
track_type	midi | audio | return | master	opt	Default regular track. Pass 'master' for master bus, 'return' for return track.

Note: A single instantaneous sample, not a running measurement. To judge loudness over time, record and analyse instead.

Transport, Tempo & Global Feel

Everything that affects the whole Set rather than one track: playback, tempo, swing, groove, recording modes and the global mixer controls.

`live_update_set`

LIGHT LIVE: YES

The master control panel in one call: tempo, time signature, the global scale root and mode, Arrangement locators, the metronome, and launch quantization.

Machine description: *Update Live Set tempo, time signature, scale, or locators (create/delete/rename).*

Argument	Type		What it's for
`clip_trigger_quantization`	integer	opt	Global clip launch quantization (0=None, 1=8 Bars, 2=4 Bars, 3=2 Bars, 4=1 Bar, 5=1/2, 6=1/2T, 7=1/4, 8=1/4T, 9=1/8, 10=1/8T, 11=1/16, 12=1/16T, 13=1/32).
`locator_id`	string	opt	locator-0 style id.
`locator_name`	string	opt	—
`locator_operation`	create | delete | rename	opt	—
`locator_time`	string	opt	Bar|beat (1|1) or beats.
`metronome`	boolean	opt	Turn metronome on (true) or off (false).
`scale`	string	opt	Root + name e.g. 'D Dorian'. Empty string disables scale.
`tempo`	number	opt	BPM 20-999.
`time_signature`	string	opt	N/D e.g. 4/4.

Note: Setting the global scale is how you get Live 12's scale-aware editing to agree with what Genius is writing.

`live_playback`

FEATHER LIVE: YES

The transport buttons. Start and stop the Arrangement, fire a specific clip or a whole scene, set the loop region, or stop all Session clips.

Machine description: *Transport and launch: play/stop arrangement, fire clip/scene, loop region, stop session clips.*

Argument	Type		What it's for
`action`	play | stop | continue | play-arrangement | update-arrangement | fire_clip | play-session-clips | stop_clip | stop-session-clips | stop-all-session-clips | fire_scene | play-scene	req	—
`loop`	boolean	opt	—
`loop_end`	string	opt	—
`loop_start`	string	opt	—
`scene_number`	integer	opt	—
`slot_number`	integer	opt	—
`start_locator`	string	opt	—

Argument	Type		What it's for
`start_time`	string	opt	—
`track_name`	string	opt	Track name instead of track_number -- exact match preferred, falls back to a unique substring match. Ignored if track_number is also given.
`track_number`	integer	opt	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: Firing a scene respects your launch quantization, so the change may not be audible until the next bar line.

`live_undo`

FEATHER LIVE: YES

Presses Live's own Undo. Because it's Live's native undo, it rolls back whatever the last change was, whether a human or Genius made it.

Machine description: *Undo the last action in Ableton Live (equivalent to Edit -> Undo).*

Arguments: none — just call it.

Note: Your main safety net. Note that one Genius action can expand into several Live undo steps, so you may need to call it more than once.

`live_redo`

FEATHER LIVE: YES

Presses Live's own Redo, reinstating something you just undid.

Machine description: *Redo the last undone action in Ableton Live (equivalent to Edit -> Redo).*

Arguments: none — just call it.

Note: Same caveat as undo — one logical change can be several steps.

`live_jump_to_time`

FEATHER LIVE: YES

Moves the Arrangement playhead to a given beat without starting playback.

Machine description: *Jump the arrangement playhead to an exact beat position without starting playback.*

Argument	Type		What it's for
`time`	number	req	Arrangement beat position (>= 0.0).

Note: Handy for 'take me to the drop' navigation when combined with locator reads.

`live_set_overdub`

FEATHER LIVE: YES

Toggles MIDI and Arrangement overdub, i.e. whether new recording is layered on top of what's already in the clip or replaces it.

Machine description: *Enable or disable MIDI/arrangement overdub recording in Ableton Live.*

Argument	Type		What it's for
`overdub`	boolean	req	True to turn overdub on, False to turn off.
`session_record`	boolean	opt	Optional: also set session record overdub state.

Note: Forgetting this is the usual reason a recording pass wipes the take before it.

`live_tap_tempo`

FEATHER LIVE: YES

Sends a tempo tap, exactly like clicking Live's TAP button. Several taps in a row set the BPM by feel.

Machine description: *Tap the session tempo, updating the Live Set BPM according to tap timing.*

Arguments: none — just call it.

> **Note:** Only meaningful when called repeatedly; a single tap does almost nothing.

`live_continue_playing`

FEATHER LIVE: YES

Resumes playback from wherever the playhead currently sits, instead of jumping back to the start.

Machine description: *Resume transport playback from the current playhead position without restarting.*

Arguments: none — just call it.

> **Note:** The difference between Live's Play and Continue behaviour.

`live_scrub`

FEATHER LIVE: YES

Nudges the playhead forwards or backwards by a number of beats, for hunting down a specific moment.

Machine description: *Scrub the transport playhead position forward or backward by a delta in beats.*

Argument	Type		What it's for
`beats`	number	req	Delta in beats (e.g. 4.0, -4.0, 16.0).

> **Note:** Accepts negative values to scrub backwards.

`live_jump_cue`

FEATHER LIVE: YES

Jumps to the next or previous locator marker in the Arrangement.

Machine description: *Step transport playhead to the next or previous cue point / locator in the Arrangement.*

Argument	Type		What it's for
`direction`	next | prev	opt	Direction to jump (default 'next').

> **Note:** Only useful if the Set actually has locators; create them with live_update_set.

`live_set_punch`

FEATHER LIVE: YES

Turns Arrangement punch-in and punch-out on or off, so recording only happens inside the loop brace.

Machine description: *Toggle arrangement punch-in and/or punch-out recording markers.*

Argument	Type		What it's for
`punch_in`	boolean	opt	Enable or disable punch-in.
`punch_out`	boolean	opt	Enable or disable punch-out.

> **Note:** Pair with live_set_loop_region to define exactly where the punch window sits.

`live_set_cue_volume`

FEATHER LIVE: YES

Sets the headphone/cue preview volume — the level you audition browser samples at, not the master output.

Machine description: *Set the master headphone / cue preview output volume (0.0 to 1.0).*

Argument	Type		What it's for
`volume`	number	req	Cue volume level (0.0 to 1.0, 0.85 ≈ 0 dB).

> **Note:** Affects monitoring only; it will not change your render.

live_stop_all_clips

FEATHER LIVE: YES

Stops every playing Session clip at once, the same as Live's global stop button.

Machine description: *Immediately stop all playing clips across all session tracks simultaneously.*

Arguments: none — just call it.

> **Note:** Respects launch quantization, so it may wait for the bar line.

live_batch_tracks

LIGHT LIVE: YES

Sweeping mixer housekeeping across the whole Set in one call: unarm everything, unmute everything, unsolo everything, or solo exactly one track exclusively.

Machine description: *Perform batch track operations across the entire Live Set (unarm_all, unmute_all, unsolo_all, solo_exclusive).*

Argument	Type		What it's for
`action`	unarm_all | unmute_all | unsolo_all | solo_exclusive	req	Batch operation to execute.
`track_number`	integer	opt	Target track number, exactly as Live shows it (required for solo_exclusive).

> **Note:** Touches every track, so it is one call but many underlying changes — and correspondingly many undo steps.

live_set_crossfader

FEATHER LIVE: YES

Sets the master crossfader position and assigns tracks to the A or B side (or neither).

Machine description: *Control master crossfader slider position (-1.0 to 1.0) and assign tracks to crossfader sides (A, B, or None).*

Argument	Type		What it's for
`assign`	A | B | None	opt	Assign track to Crossfade side A, B, or None.
`track_number`	integer	opt	Optional track number to assign, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`value`	number	opt	Master crossfader position: -1.0 (full A), 0.0 (center), 1.0 (full B).

> **Note:** A track assigned to neither side is unaffected by the crossfader entirely.

live_set_loop_region

FEATHER LIVE: YES

Sets the Arrangement loop brace: whether looping is on, where it starts, and how long it is, in beats.

Machine description: *Set the Arrangement loop brace: enable/disable the loop and its start/length in beats.*

Argument	Type		What it's for
`enabled`	boolean	opt	True to enable the arrangement loop, false to disable it.
`loop_length`	number	opt	Loop length in beats.
`loop_start`	number	opt	Loop start position in beats.

> **Note:** Verified live in v1.0.40. Start and length are set before the enable flag, so the loop always switches on around the correct region.

live_set_groove_amount

FEATHER LIVE: YES

Sets the global Groove Pool amount — one master dial for how strongly any applied groove pushes timing and velocity across the whole Set.

Machine description: *Set the global groove pool amount -- how strongly the applied groove affects timing/velocity across the whole Live Set.*

Argument	Type		What it's for
`amount`	number	req	0.0 (no groove) to 1.0 (full groove).

Note: 0.0 is no groove, 1.0 is full. This scales grooves already attached to clips; it does not attach one.

`live_nudge`

FEATHER LIVE: YES

Momentarily speeds up or slows down the transport, the way a DJ nudges a platter to beatmatch against an external source.

Machine description: *Momentarily nudge the transport tempo up or down, for manual beatmatching against an external source.*

Argument	Type		What it's for
`direction`	up | down	req	—

Note: Temporary by design — it does not change the stored BPM.

`live_set_swing`

FEATHER LIVE: YES

Sets the global swing amount applied when recording or playing back quantized material.

Machine description: *Set the global swing amount applied to quantized recording/playback.*

Argument	Type		What it's for
`amount`	number	req	0.0 (no swing) to 1.0 (max swing).

Note: This is the record/playback swing, distinct from a groove baked into a clip with live_commit_groove.

`live_set_automation_record`

FEATHER LIVE: YES

Turns Session View automation recording on or off — whether knob moves you make during playback get captured into the clip.

Machine description: *Enable or disable Session View automation recording.*

Argument	Type		What it's for
`enabled`	boolean	req	—

Note: Live does not report this property back reliably within a single call, so v1.0.40 made the tool trust the write instead of failing a bogus read-back check.

`live_back_to_arranger`

FEATHER LIVE: YES

Presses Live's Back to Arrangement button, handing playback control back to the Arrangement after Session clips have overridden it.

Machine description: *Trigger Ableton's Back to Arrangement button to return playback control to the Arrangement timeline.*

Arguments: none — just call it.

Note: The fix for 'my Arrangement won't play what I can see on the timeline'.

`live_re_enable_automation`

FEATHER LIVE: YES

Re-arms automation that you overrode by grabbing a knob during playback, the same as Live's Re-Enable Automation button.

Machine description: *Re-enable parameter automation that was overridden by manual control tweaks (the orange automation button in Live).*

Arguments: none — just call it.

> **Note:** If your automation mysteriously stopped moving, this is almost always the answer.

Tracks & Mixer

Creating, reading, renaming and reshaping tracks, plus the per-track mixer.

`live_read_track`

MEDIUM LIVE: YES

The full dossier on one track: its mixer settings, every device on it, every clip in it, its drum map if it has one, and its input/output routing.

Machine description: *Read one track (midi/audio/return/master): mixer, devices, session clips, arrangement clips, drum map, routings.*

Argument	Type		What it's for
`track_number`	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`include`	array: session-clips | arrangement-clips | notes | devices | drum-map | routings | available-routings | mixer | color | *	opt	—
`track_name`	string	opt	Track name instead of track_number -- exact match preferred, falls back to a unique substring match. Ignored if track_number is also given.
`track_type`	midi | audio | return | master	opt	—

Note: Far cheaper than live_read_set when you only care about one track.

`live_update_track`

LIGHT LIVE: YES

The per-track everything-call: name, colour, mute/solo/arm, volume and pan, send levels, monitoring mode and routing. Twenty-one optional settings, all in one action.

Machine description: *Update track name, color, mute/solo/arm, volume/pan, send, monitoring, routing.*

Argument	Type		What it's for
`track_number`	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`arm`	boolean	opt	—
`color`	string	opt	—
`crossfade_assign`	none | a | b	opt	Master crossfader assignment.
`delay_ms`	number	opt	Track delay compensation offset in milliseconds.
`gain_db`	number	opt	—
`input_routing_channel`	string	opt	—
`input_routing_type`	string	opt	—
`monitoring`	in | auto | off	opt	Track monitoring state (in, auto, off).
`monitoring_state`	in | auto | off	opt	—
`mute`	boolean	opt	—
`name`	string	opt	—

Argument	Type		What it's for
output_routing_channel	string	opt	—
output_routing_type	string	opt	—
pan	number	opt	—
send	string	opt	Return letter A/B/C…
send_gain	number	opt	0-1 send amount.
send_gain_db	number	opt	—
solo	boolean	opt	—
track_name	string	opt	Track name instead of track_number -- exact match preferred, falls back to a unique substring match. Ignored if track_number is also given. This selects the target track; use `name` to rename it.
track_type	midi | audio | return | master	opt	—
volume	number	opt	0-1 fader (0.85 ≈ 0 dB).

Note: Because it takes so many optional fields, Genius can do a whole channel-strip setup in a single call rather than ten.

live_create_track

LIGHT LIVE: YES

Adds new MIDI, audio or return tracks to the Set.

Machine description: *Create midi, audio, or return track(s).*

Argument	Type		What it's for
type	midi | audio | return	req	—
arm	boolean	opt	—
color	string	opt	—
count	integer	opt	—
mute	boolean	opt	—
name	string	opt	Name, or comma-separated for count>1.
position_number	integer	opt	Insert position, exactly as Live would number the new track (e.g. 1 = before today's track 1). Omit to append at the end.
solo	boolean	opt	—

Note: Can create several at once. New tracks land at the end unless you say otherwise, which shifts nothing else.

live_edit_track

LIGHT LIVE: HYBRID

The tidy three-in-one for whole-track operations: delete it, duplicate it, or rename it.

Machine description: *Unified track editor. op is 'delete', 'duplicate', or 'rename'. 'rename' requires name. 'duplicate' inserts copy directly after source.*

Argument	Type		What it's for
op	delete | duplicate | rename	req	Operation to perform.
track_number	integer	req	Target track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Argument	Type		What it's for
`name`	string	opt	New track name (required for op='rename').

Note: A convenience wrapper that fans out to the right underlying call — less for the model to get wrong.

`live_delete`

FEATHER LIVE: YES

The general-purpose remover: deletes a track, a clip, a scene or a device, whichever you name.

Machine description: *Delete a track, session clip, arrangement clip, scene, or device.*

Argument	Type		What it's for
`type`	track | clip | arrangement_clip | scene | device | drum-pad	req	—
`clip_number`	integer	opt	For arrangement_clip: number in track.arrangement_clips, exactly as Live shows it.
`device_number`	integer	opt	—
`path`	string	opt	—
`scene_number`	integer	opt	—
`slot_number`	integer	opt	—
`track_number`	integer	opt	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`track_type`	midi | audio | return | master	opt	—

Note: Destructive. Recoverable with live_undo, but only immediately — verify the target number first.

`live_duplicate`

LIGHT LIVE: YES

The general-purpose copier: duplicates a track, a scene or a Session clip.

Machine description: *Duplicate a track, scene, session clip, or duplicate a session clip into the Arrangement timeline.*

Argument	Type		What it's for
`type`	track | clip | scene | device	req	—
`destination_time`	number	opt	Beat position in Arrangement timeline. When provided for type=clip, duplicates the Session clip directly into the Arrangement.
`scene_number`	integer	opt	—
`slot_number`	integer	opt	—
`to_slot`	integer	opt	—
`to_track_number`	integer	opt	Optional target track number when duplicating to Arrangement.
`track_number`	integer	opt	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: Duplicating a track copies its devices too, which on a plugin-heavy track costs real CPU and RAM in Live.

`live_select`

FEATHER LIVE: YES

Drives Live's GUI selection — highlights a track, scene, clip slot or device, and can switch which view you're looking at.

Machine description: *Show/select a track, scene, clip slot, or device. Switch Session/Arrangement. Open a plugin window already on a track.*

Argument	Type		What it's for
`device_path`	string	opt	—
`open_plugin_window`	boolean	opt	—
`path`	string	opt	—
`scene_number`	integer	opt	—
`slot_number`	integer	opt	Session clip slot number to select, exactly as Live shows it -- combine with track_number.
`track_name`	string	opt	Track name instead of track_number -- exact match preferred, falls back to a unique substring match. Ignored if track_number is also given.
`track_number`	integer	opt	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`track_type`	midi | audio | return | master	opt	—
`view`	session | arrangement	opt	—

Note: Purely visual. Use it to show you what Genius is about to work on before it commits.

`live_set_fold_state`

FEATHER LIVE: YES

Reads or toggles whether a Group track is collapsed or expanded.

Machine description: *Inspect or toggle the fold/collapse state of a Group track in the Live mixer/session.*

Argument	Type		What it's for
`track_number`	integer	req	Group track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`folded`	boolean	opt	True to collapse/fold group; False to expand/unfold.

Note: Only meaningful on Group tracks; harmless everywhere else.

Scenes

Session View rows — reading, editing and capturing them.

live_read_scene

LIGHT LIVE: YES

Reads one scene and the clips sitting in it across all tracks.

Machine description: *Read a scene and its clips.*

Argument	Type		What it's for
scene_number	integer	req	—
include	array: clips | notes | color | *	opt	—

Note: The horizontal counterpart to live_read_track.

live_update_scene

FEATHER LIVE: YES

Sets a scene's name, colour, or its own tempo (so launching it changes the BPM).

Machine description: *Update scene name, color, or tempo.*

Argument	Type		What it's for
color	string	opt	—
name	string	opt	—
scene_number	integer	opt	—
scene_numbers	array of integer	opt	Scene numbers to update, exactly as Live shows them, e.g. [1,3]. Updates one scene when scene_number is given instead.
tempo	number	opt	—

Note: Scene tempo is how you build a Set that changes speed as you move down the Session grid.

live_create_scene

LIGHT LIVE: YES

Makes a new empty scene, or captures whatever is playing right now into a brand new scene.

Machine description: *Create empty scene(s) or capture playing session clips.*

Argument	Type		What it's for
capture	boolean	opt	Copy currently playing session clips.
color	string	opt	—
count	integer	opt	—
name	string	opt	—
scene_number	integer	opt	—

Note: The capture mode is the fastest way to freeze a good improvised combination before you lose it.

live_edit_scene

LIGHT LIVE: HYBRID

Delete, duplicate or rename a scene, in one unified call.

Machine description: *Unified scene editor. op is 'delete', 'duplicate', or 'rename'. 'rename' requires name.*

Argument	Type		What it's for
`op`	delete | duplicate | rename	**req**	Operation to perform.
`scene_number`	integer	**req**	Target scene number.
`name`	string	opt	New scene name (required for op='rename').

Note: Same convenience-wrapper pattern as live_edit_track.

Session Clips — Create & Edit

The clip-level workhorses: making clips, writing notes into them, and shaping how they loop, warp and launch.

`live_read_clip`

MEDIUM LIVE: YES

Opens up a Session clip and reports what's inside: every MIDI note with its timing, plus loop and warp settings for audio.

Machine description: *Read a session clip: notes, timing, warp/sample for audio.*

Argument	Type		What it's for
`include`	array: notes | timing | sample | warp | color | *	opt	—
`slot_number`	integer	opt	Session clip slot number.
`track_name`	string	opt	Track name instead of track_number -- exact match preferred, falls back to a unique substring match. Ignored if track_number is also given.
`track_number`	integer	opt	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: A dense MIDI clip means a big note list — this is one of the larger reads in normal use.

`live_update_clip`

LIGHT LIVE: YES

The clip equivalent of live_update_track, with thirty-two optional settings: name, colour, mute, loop points, length, quantization, and for audio clips the warp mode and pitch.

Machine description: *Update clip name/color/mute/loop/length, write notes, quantize, or audio warp/pitch.*

Argument	Type		What it's for
`color`	string	opt	—
`end_marker`	number	opt	End marker in beats.
`fade_in_end`	number	opt	Fade-in end position in beats (audio clips).
`fade_in_start`	number	opt	Fade-in start position in beats (audio clips).
`fade_out_end`	number	opt	Fade-out end position in beats (audio clips).
`fade_out_start`	number	opt	Fade-out start position in beats (audio clips).
`follow_action_a`	none | stop | again | previous | next | first | last | any | other | jump	opt	Follow action A.
`follow_action_b`	none | stop | again | previous | next | first | last | any | other | jump	opt	Follow action B.
`follow_action_chance`	number	opt	Probability of follow action A (0.0–1.0).
`follow_action_enabled`	boolean	opt	Whether follow actions are active.
`follow_action_time`	number	opt	Follow action trigger time in beats.
`gain`	number	opt	Audio clip gain (dB or float value).
`launch_mode`	trigger | gate | toggle | repeat	opt	Clip launch mode.

Argument	Type		What it's for
launch_quantization	string	opt	global, none, 8_bars, 4_bars, 2_bars, 1_bar, 1/2, 1/4, 1/8, 1/16, 1/32...
legato	boolean	opt	Legato (continuous) launch mode.
length	string	opt	e.g. 4bar or n/4.
length_beats	number	opt	—
loop	boolean	opt	—
mute	boolean	opt	—
name	string	opt	—
notes	string	opt	Pal-style notes (v100 n/16 C1 1|1) or JSON text of a note array.
pitch	number	opt	Audio clip pitch transposition in semitones (-48 to +48).
pitch_shift	number	opt	—
quantize	number	opt	—
ram_mode	boolean	opt	Audio clip RAM mode toggle.
replace	boolean	opt	—
slot_number	integer	opt	Session clip slot number.
start_marker	number	opt	Start marker in beats.
track_number	integer	opt	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
velocity_amount	number	opt	Velocity sensitivity (0.0 to 1.0).
warp_mode	beats | tones | texture | re-pitch | complex | complex_pro	opt	Audio warp mode.
warping	boolean	opt	—

Note: The widest single action in the toolset. Genius can reshape a clip completely without a second call.

live_create_clip

LIGHT LIVE: YES

Creates a new MIDI or audio clip, either in a Session slot or directly on the Arrangement timeline.

Machine description: *Create a MIDI or audio clip in Session (slot) or Arrangement (arrangement_start). Audio: pass sample_file.*

Argument	Type		What it's for
arrangement_start	string	opt	Bar|beat in arrangement.
auto	play-clip | play-scene	opt	—
color	string	opt	—
length	string	opt	—
length_beats	number	opt	—
looping	boolean	opt	—
name	string	opt	—
notes	string	opt	MIDI notes after create (Pal-style string or JSON array text).
sample_file	string	opt	Absolute audio path for an audio clip.

Argument	Type		What it's for
slot_number	integer	opt	Session clip slot number.
track_name	string	opt	Track name instead of track_number -- exact match preferred, falls back to a unique substring match. Ignored if track_number is also given.
track_number	integer	opt	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: For audio it needs a real file path on disk; find one first with live_find_samples or live_library.

live_edit_clip

LIGHT LIVE: HYBRID

Delete, duplicate or rename a clip, in one unified call.

Machine description: *Unified clip editor. op is 'delete', 'duplicate', or 'rename'. 'rename' requires name. Duplicate with no target places into next empty slot on same track.*

Argument	Type		What it's for
op	delete | duplicate | rename	req	Operation to perform.
slot_number	integer	req	Clip slot number.
track_number	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
name	string	opt	New clip name (required for op='rename').
target_slot_number	integer	opt	Optional destination slot number for op='duplicate'.

Note: Same wrapper pattern as the track and scene versions.

live_write_notes

MEDIUM LIVE: YES

Writes MIDI notes into a Session clip, creating the clip if the slot is empty. This is the fundamental 'put music in Ableton' action that most generative prompts eventually land on.

Machine description: *Write MIDI notes into a session clip (creates the clip if empty). Prefer this for drum patterns, chords, bass, melody.*

Argument	Type		What it's for
notes	array of object	req	—
slot_number	integer	req	—
track_number	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
length_beats	number	opt	—
name	string	opt	—
replace	boolean	opt	—
track_name	string	opt	Track name instead of track_number -- exact match preferred, falls back to a unique substring match. Ignored if track_number is also given.

Note: Cost scales with how many notes you write. A long dense pattern is a genuinely large message across the bridge.

live_remove_notes

LIGHT LIVE: YES

Surgically deletes notes inside a specific pitch and time window, leaving everything outside it untouched.

Machine description: *Surgically remove MIDI notes from a clip within a specific time and pitch window without erasing other notes in the clip.*

Argument	Type		What it's for
`from_pitch`	integer	opt	Lowest MIDI pitch to remove (0-127, default 0).
`from_time`	number	opt	Start beat (default 0.0).
`pitch_span`	integer	opt	Number of semitones above from_pitch to clear (default 128).
`slot_number`	integer	opt	Session clip slot number.
`time_span`	number	opt	Duration in beats to clear (default entire clip length).
`track_number`	integer	opt	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: The precise alternative to clearing a clip — e.g. remove only the kicks in bars 5-8.

`live_duplicate_loop`

LIGHT LIVE: YES

Doubles a clip's loop length and copies the existing notes into the new half, exactly like Live's Duplicate Loop button.

Machine description: *Double a clip's loop length and duplicate all notes inside it (equivalent to Live's native Duplicate Loop button).*

Argument	Type		What it's for
`slot_number`	integer	req	Clip slot number.
`track_number`	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: The standard way to turn a 1-bar idea into 2 bars before varying the second half.

`live_commit_groove`

LIGHT LIVE: YES

Permanently bakes groove and quantization into a clip's actual note positions, rather than leaving it as a playback effect. You choose the grid and how strongly to apply it.

Machine description: *Commit (bake) groove/quantize permanently into a clip's MIDI notes. quantize_to sets the grid (1=1/4, 5=1/16, 8=1/32, default 5); amount 0.0–1.0 blends original vs full groove.*

Argument	Type		What it's for
`slot_number`	integer	req	Slot number, exactly as Live shows it.
`track_number`	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`amount`	number	opt	Quantize amount 0.0–1.0 (default 1.0).
`quantize_to`	integer	opt	Grid: 1=1/4, 2=1/8, 3=1/8T, 4=1/8+T, 5=1/16, 6=1/16T, 7=1/16+T, 8=1/32 (default 5=1/16).

Note: Destructive to the original timing. The amount control blends between what you had and full quantization.

`live_groove_pool`

LIGHT LIVE: YES

Inspects Live's Groove Pool and can attach one of its grooves to a clip.

Machine description: *Inspect grooves loaded in the Groove Pool, or attach a groove to a clip to apply swing and timing feel.*

Argument	Type		What it's for
`clear`	boolean	opt	Set true to remove groove from clip.
`clip_number`	integer	opt	Optional clip slot number.

Argument	Type		What it's for
groove_number	integer	opt	Groove number from the Groove Pool to attach.
track_number	integer	opt	Optional track number to attach groove to, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: Attaching is non-destructive and scaled globally by live_set_groove_amount; baking it in is live_commit_groove.

live_set_clip_fades

FEATHER LIVE: YES

Intended to set audio-clip fade-in and fade-out timing.

Machine description: *Set fade in/out times (in beats) on an audio clip. All four points are optional; omit any to leave unchanged.*

Argument	Type		What it's for
slot_number	integer	req	Slot number, exactly as Live shows it.
track_number	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
fade_in_end	number	opt	Fade-in end position in beats.
fade_in_start	number	opt	Fade-in start position in beats.
fade_out_end	number	opt	Fade-out end position in beats.
fade_out_start	number	opt	Fade-out start position in beats.

Note: CONFIRMED INERT. The underlying Clip.fade_* properties are absent from real clips in Live 12.4 — see Book 4. This call currently does nothing.

live_set_clip_markers

FEATHER LIVE: YES

Sets a clip's start and end markers and its loop start and end — which slice of the material plays, and which slice repeats.

Machine description: *Set start/end markers and/or loop start/end times on a clip (in beats).*

Argument	Type		What it's for
slot_number	integer	req	Slot number, exactly as Live shows it.
track_number	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
end_marker	number	opt	End marker in beats.
end_time	number	opt	Alias for loop_end (loop end time in beats).
loop_end	number	opt	Loop end time in beats.
loop_start	number	opt	Loop start time in beats.
start_marker	number	opt	Start marker in beats.
start_time	number	opt	Alias for loop_start (loop start time in beats).

Note: The reliable way to trim a clip without editing the underlying audio or notes.

live_manage_warp_markers

LIGHT LIVE: YES

Reads, adds or deletes warp markers on an audio clip — the anchor points that tell Live how the audio should stretch to the grid.

Machine description: *Get, add, or delete warp markers on an audio clip. action='get' lists all markers; 'add' inserts at beat_time (with optional sample_time); 'delete' removes at beat_time.*

Argument	Type		What it's for
action	get | add | delete	req	Operation: get, add, or delete.
slot_number	integer	req	Slot number, exactly as Live shows it.
track_number	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
beat_time	number	opt	Warp marker position in beats (required for add/delete).
sample_time	number	opt	Sample position in seconds (optional for add).

Note: Warp markers are how you fix a sample that drifts out of time. Deleting the wrong one can badly smear the audio.

live_set_clip_follow_action

FEATHER LIVE: YES

Intended to set a clip's follow actions — what plays next, with what probability, after what time.

Machine description: *Set follow actions for a Session clip: action A/B (none/stop/again/previous/next/first/last/any/other/jump), chance (0.0–1.0), time in beats, and enabled flag.*

Argument	Type		What it's for
slot_number	integer	req	Slot number, exactly as Live shows it.
track_number	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
action_a	none | stop | again | previous | next | first | last | any | other | jump	opt	Follow action A.
action_b	none | stop | again | previous | next | first | last | any | other | jump	opt	Follow action B.
chance	number	opt	Probability of action A (0.0–1.0, default 1.0).
enabled	boolean	opt	Whether follow actions are active.
time	number	opt	Follow action time in beats (default = clip length).

Note: CONFIRMED INERT. The follow_action_* properties are absent from real clips in this Live version — see Book 4.

live_get_clip_follow_action

FEATHER LIVE: YES

Intended to read back a clip's follow-action settings.

Machine description: *Read all follow action settings (action A/B names, chance, time, enabled) from a Session clip.*

Argument	Type		What it's for
slot_number	integer	req	Slot number, exactly as Live shows it.
track_number	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: CONFIRMED INERT, same root cause as the setter.

live_set_clip_launch_mode

FEATHER LIVE: YES

Sets how a clip responds when you trigger it: gate, trigger, toggle or repeat, its launch quantization, and whether it plays legato.

Machine description: *Set the launch mode, launch quantization, and legato flag for a Session clip.*

Argument	Type		What it's for
`slot_number`	integer	**req**	Slot number, exactly as Live shows it.
`track_number`	integer	**req**	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`launch_mode`	trigger | gate | toggle | repeat	opt	Clip launch mode.
`launch_quantization`	string	opt	global, none, 8_bars, 4_bars, 2_bars, 1_bar, 1/2, 1/4, 1/8, 1/16, 1/32.
`legato`	boolean	opt	Whether legato (continuous) launch is enabled.

Note: This is the heart of building a playable, performable Session set rather than a static one.

Clip Automation & Envelopes

Drawing parameter movement inside clips, procedurally.

live_set_clip_automation

MEDIUM LIVE: YES

Writes an automation envelope into a clip for a named parameter — you supply the list of time/value breakpoints and it draws the curve.

Machine description: *Write a clip automation envelope for a parameter (by name), on a Session clip (slot_number) or an Arrangement clip (arrangement_clip_number). envelope_data is a list of {time, value} points.*

Argument	Type		What it's for
envelope_data	array of object	req	List of {time (beats), value} automation points.
parameter_name	string	req	Parameter name (e.g. 'Volume', 'Dry/Wet', 'Filter Freq').
track_number	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
arrangement_clip_number	integer	opt	Arrangement clip number in track.arrangement_clips, exactly as Live shows it, instead of a Session clip slot_number.
device_number	integer	opt	Device number (default 1) for device parameter lookup.
slot_number	integer	opt	Session clip slot number, exactly as Live shows it. Ignored if arrangement_clip_number is given.
track_name	string	opt	Track name instead of track_number -- exact match preferred, falls back to a unique substring match. Ignored if track_number is also given.

Note: Confirmed working end to end in v1.0.40. Cost scales with the number of points.

live_get_clip_automation

FEATHER LIVE: YES

Checks whether a clip has any automation envelopes on it.

Machine description: *Check whether a clip has automation envelopes.*

Argument	Type		What it's for
track_number	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
arrangement_clip_number	integer	opt	Arrangement clip number in track.arrangement_clips, exactly as Live shows it, instead of a Session clip slot_number.
slot_number	integer	opt	Session clip slot number, exactly as Live shows it. Ignored if arrangement_clip_number is given.

Note: Cheap pre-flight check before you overwrite something the user drew by hand.

live_add_shaped_automation

MEDIUM LIVE: HYBRID

Generates a mathematical curve — sine, triangle, saw, square or random — and writes it in as an envelope. You say how many cycles fit across the clip and the value range to sweep.

Machine description: *Generate a shaped automation curve (sine/triangle/saw/square/random) and write it as a clip envelope for a named parameter. Cycles controls how many waveform cycles fit in the clip length.*

Argument	Type		What it's for
parameter_name	string	req	Parameter name to automate (e.g. 'Volume', 'Dry/Wet').

Argument	Type		What it's for
`shape`	sine | triangle | saw | square | random	req	Waveform shape.
`slot_number`	integer	req	Slot number, exactly as Live shows it.
`track_number`	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`cycles`	number	opt	Number of waveform cycles over the clip length (default 1.0).
`device_number`	integer	opt	Device number (default 1).
`length`	number	opt	Clip length in beats (auto-detected if omitted).
`max_value`	number	opt	Maximum parameter value (default 1.0).
`min_value`	number	opt	Minimum parameter value (default 0.0).
`points`	integer	opt	Number of automation points (default 32).
`track_name`	string	opt	Track name instead of track_number -- exact match preferred, falls back to a unique substring match. Ignored if track_number is also given.

Note: The curve is computed locally then written in one go. This is how you get an LFO-style sweep without an LFO device.

`live_clear_clip_envelopes`

LIGHT LIVE: YES

Wipes all automation and modulation envelopes off a clip.

Machine description: *Reset and clear all parameter automation and modulation envelopes inside a Session clip or Arrangement clip via native LOM clear_all_envelopes.*

Argument	Type		What it's for
`track_number`	integer	req	Target track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`arrangement_clip_number`	integer	opt	Arrangement clip number (for arrangement timeline clips).
`slot_number`	integer	opt	Session clip slot number (for session clips).

Note: Destructive and broad — it does not discriminate between envelopes you wrote and ones you drew.

Arrangement Timeline

Everything on the horizontal timeline: placing clips, reading and writing their notes, moving them around, and recording into them.

`live_record_to_arrangement`

VERY HEAVY LIVE: YES

Records a Session scene's performance down into the Arrangement, the way you'd capture a live jam into a finished timeline.

Machine description: *Record a Session scene's performance into the Arrangement timeline (arms Song's Arrangement Record, fires the scene, so it plays and records the same as pressing Record then launching a scene by hand). Different from live_create_scene capture=true, which only snapshots currently playing clips into a NEW SESSION scene — this one writes into the Arrangement. Never blocks: with duration_beats it schedules an auto-stop and returns immediately with the estimated wait; without it, recording keeps going until you call this again with op='stop'.*

Argument	Type		What it's for
`duration_beats`	number	opt	Auto-stop after this many beats at the Set's current tempo. Omit to leave it running until op=stop.
`op`	start | stop	opt	Default start.
`scene_number`	integer	opt	Required for op=start.
`stop_playback`	boolean	opt	op=stop only: also stop the transport, not just disarm recording.

Note: Runs in real time — recording 32 bars genuinely takes 32 bars of wall-clock time.

`live_duplicate_to_arrangement`

LIGHT LIVE: YES

Copies a Session clip onto the Arrangement timeline at a specific beat position.

Machine description: *Directly duplicate a Session-view clip into the Arrangement timeline at a specific beat position via track.duplicate_clip_to_arrangement.*

Argument	Type		What it's for
`destination_time`	number	req	Beat position in Arrangement timeline (e.g. 0.0, 16.0, 32.0).
`slot_number`	integer	opt	Source clip slot number.
`to_track_number`	integer	opt	Target track number (defaults to same track).
`track_number`	integer	opt	Source track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: The non-real-time alternative to recording. This is how you assemble an arrangement instantly from Session ideas.

`live_read_arrangement_clips`

LIGHT LIVE: YES

Lists every clip placed on a track's Arrangement timeline, with positions.

Machine description: *Read all clips placed along the Arrangement timeline for a track (start_time, end_time, length, name, color, is_midi, is_audio, muted).*

Argument	Type		What it's for
`track_number`	integer	req	Track number to inspect, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`track_type`	midi | audio | return | master	opt	—

Note: Needed before moving anything, because Arrangement clips are addressed by their index in this list.

live_update_arrangement_clip

FEATHER LIVE: YES

Renames, recolours or mutes a clip sitting on the Arrangement timeline.

Machine description: *Update or rename a clip situated on the Arrangement timeline.*

Argument	Type		What it's for
clip_number	integer	req	Number within track.arrangement_clips, exactly as Live shows it (ordered by start_time).
track_number	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
color	string	opt	Hex color code.
mute	boolean	opt	Mute state.
name	string	opt	New clip name.

Note: Colour-coding your Arrangement sections is the cheapest navigation upgrade there is.

live_move_arrangement_clip

LIGHT LIVE: YES

Moves an Arrangement clip to a new time position, and optionally onto a different track.

Machine description: *Move an arrangement clip to a new beat start time and/or different track on the timeline.*

Argument	Type		What it's for
new_start_time	number	req	New beat start position.
source_clip_number	integer	req	Source arrangement clip number.
source_track_number	integer	req	Source track number.
target_track_number	integer	opt	Optional target track number (defaults to source track).

Note: Moving across tracks only works between compatible types — MIDI to MIDI, audio to audio.

live_read_arrangement_notes

MEDIUM LIVE: YES

Reads the MIDI notes out of a clip on the Arrangement timeline.

Machine description: *Read all MIDI notes directly from an arrangement clip on the timeline.*

Argument	Type		What it's for
clip_number	integer	req	Arrangement clip number.
track_number	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: The Arrangement counterpart of live_read_clip; same payload considerations.

live_write_arrangement_notes

MEDIUM LIVE: YES

Writes or appends MIDI notes into an Arrangement clip.

Machine description: *Write or append MIDI notes directly into an arrangement clip on the timeline.*

Argument	Type		What it's for
clip_number	integer	req	Arrangement clip number.

Argument	Type		What it's for
`notes`	array of object	req	List of note objects or JSON string.
`track_number`	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`replace`	boolean	opt	True to replace all notes (default), False to append.

Note: Cost scales with note count, same as the Session version.

`live_remove_arrangement_notes`

LIGHT LIVE: YES

Surgically removes notes from an Arrangement clip within a pitch and time window.

Machine description: *Surgically remove MIDI notes in a pitch/time range from an arrangement clip on the timeline without erasing the whole clip.*

Argument	Type		What it's for
`clip_number`	integer	req	Arrangement clip number.
`track_number`	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`from_pitch`	integer	opt	Lowest pitch (0-127, default 0).
`from_time`	number	opt	Start time in beats (default 0).
`pitch_span`	integer	opt	Number of semitones span (default 128).
`time_span`	number	opt	Duration in beats (default 100000).

Note: Same targeted behaviour as live_remove_notes, applied to the timeline.

`live_read_arrangement_length`

FEATHER LIVE: YES

Reports the total length of the Arrangement in beats — effectively, how long the song is.

Machine description: *Query the total length of the Arrangement song timeline in beats (from song.song_length).*

Arguments: none — just call it.

Note: Useful as a sanity check that a build actually produced a full-length track.

`live_capture_midi`

LIGHT LIVE: YES

Pulls Live's retrospective MIDI capture buffer into a new clip — the 'I just played something good but wasn't recording' rescue button.

Machine description: *Trigger Ableton's native Capture MIDI feature to retrieve recently played MIDI into a new clip on the armed track.*

Arguments: none — just call it.

Note: Only works if Live actually has something in its capture buffer; it cannot invent a performance.

Take Lanes & Comping

Multiple takes stacked under one track, for comping the best parts together.

`live_create_take_lane`

FEATHER LIVE: YES

Adds a new take lane, audio or MIDI, underneath a track.

Machine description: *Create an audio or MIDI take lane on a track for comping multiple recorded passes.*

Argument	Type		What it's for
`track_number`	integer	req	Target track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: Take lanes are where repeated recording passes stack up for later comping.

`live_read_take_lanes`

FEATHER LIVE: YES

Lists a track's existing take lanes with their index and name.

Machine description: *Get all take lanes on a track with their index and name.*

Argument	Type		What it's for
`track_number`	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: Read this first — take lanes are addressed by index.

`live_update_take_lane`

FEATHER LIVE: YES

Renames an existing take lane.

Machine description: *Rename an existing take lane on an arrangement track.*

Argument	Type		What it's for
`name`	string	req	New name for the take lane.
`take_lane_number`	integer	req	Take lane number.
`track_number`	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: Naming takes ('bright', 'gritty', 'safe') is what makes comping later actually navigable.

`live_create_take_lane_clip`

LIGHT LIVE: YES

Creates an empty MIDI clip inside a specific take lane.

Machine description: *Create an empty MIDI clip directly inside a take lane on an arrangement track.*

Argument	Type		What it's for
`take_lane_number`	integer	req	Take lane number.
`track_number`	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`length`	number	opt	Clip length in beats (default 4.0).
`start_time`	number	opt	Beat start position (default 0.0).

Note: Gives you a target to write alternative versions into without disturbing the main lane.

Devices, Racks & Macros

Instruments and effects: inspecting them, setting their parameters, loading them, and wrangling Racks and Drum Racks.

`live_read_device`

MEDIUM LIVE: YES

Reads a device in detail — its parameters and their current values — and works on native Live devices, chains inside a Rack, individual Drum Rack pads, and third-party VST/AU plugins that are actually present on this machine.

Machine description: *Read a device, rack chain, or drum pad (native or already-present VST/AU). Path examples: t1/d1, t2/d1/c1, t1/d1/pC1.*

Argument	Type		What it's for
`device_number`	integer	opt	—
`include`	array: params | param-values | chains | drum-pads | drum-map | *	opt	—
`param_search`	string	opt	—
`path`	string	opt	t1/d1 or t1/d1/pC1.
`track_name`	string	opt	Track name instead of track_number -- exact match preferred, falls back to a unique substring match. Ignored if track_number is also given.
`track_number`	integer	opt	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: On a plugin with hundreds of parameters this is a large payload. Narrow it down if you can.

`live_update_device`

LIGHT LIVE: YES

Sets device parameters, renames the device, mutes or solos Rack chains, and can load a sample onto a Drum Rack pad.

Machine description: *Set device params (display units) on a native device or an already-present VST/AU, rename, or mute/solo chains. To load a sample onto a Drum Rack pad, set a param named 'p<note>/sample' (e.g. 'pC1/sample') to an absolute file path from live_find_samples or live_library -- this creates the pad's chain and a Simpler on it if needed, then loads the file (requires Live 12.3+ for the pad, 12.4+ to actually load the sample; check the response for params_skipped and tell the user to drag it in by hand only if that pad is listed there).*

Argument	Type		What it's for
`device_number`	integer	opt	—
`is_active`	boolean	opt	—
`is_collapsed`	boolean	opt	Collapse or expand the device in Detail View.
`mute`	boolean	opt	—
`name`	string	opt	—
`params`	array of object	opt	—
`path`	string	opt	—
`solo`	boolean	opt	—
`track_name`	string	opt	Track name instead of track_number -- exact match preferred, falls back to a unique substring match. Ignored if track_number is also given.
`track_number`	integer	opt	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: As of v1.0.40 this writes the display value directly where possible, so values land exactly instead of converging to the nearest displayed step.

live_create_device

LIGHT LIVE: YES

Inserts a native Ableton device onto a track by name.

Machine description: *Insert a native Live device onto a track by name (Drum Rack, Drift, Reverb, Scale, Simpler…), Live 12.3+ only. Omit device_name to list insertable native device names. Cannot add VST/AU, Max for Live devices, or browser presets. For a Drum Rack, bundle pad samples into the same call via params: 'p<note>/sample' = an absolute file path (see live_update_device).*

Argument	Type		What it's for
track_number	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
device_name	string	opt	e.g. Drum Rack, Drift, Reverb, EQ Eight, Scale, Arpeggiator.
params	array of object	opt	—
track_name	string	opt	Track name instead of track_number -- exact match preferred, falls back to a unique substring match. Ignored if track_number is also given.

Note: Live 12.3+ only, and native devices only — it cannot load third-party plugins. See Book 4.

live_move_device

FEATHER LIVE: YES

Repositions a device earlier or later in a track's effect chain.

Machine description: *Move a device to a new position index within a track's device chain.*

Argument	Type		What it's for
device_number	integer	req	Current device number in the chain.
new_device_number	integer	req	Target position in the chain, exactly as Live shows it (1 = first).
track_number	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: Order matters enormously: compressor-then-reverb and reverb-then-compressor are completely different sounds.

live_read_macros

LIGHT LIVE: YES

Lists a Rack's macro controls — their names, current values, ranges, and whether they're stepped.

Machine description: *List the macro controls of a Rack device (Instrument, Audio Effect, MIDI, or Drum Rack) on a track, including names, values, ranges, is_quantized, and is_enabled state.*

Argument	Type		What it's for
device_number	integer	opt	—
path	string	opt	Path e.g. t1/d1 or t2/d1.
track_number	integer	opt	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
track_type	midi | audio | return | master	opt	—

Note: Read before writing so you know what Macro 3 actually is on this particular Rack.

live_update_macro

FEATHER LIVE: YES

Sets a Rack macro by number or by name, in either the device's own units or a plain 0-1 range.

Machine description: *Set a Rack macro control value by number (1-based, e.g. macro_number 1 = Macro 1) or name. Accepts native units (value) or normalized 0.0-1.0 (value_normalized).*

Argument	Type		What it's for
device_number	integer	opt	—
macro	string	opt	Macro name, e.g. 'Filter Freq'. For numeric addressing use macro_number instead.
macro_number	integer	opt	Macro number, exactly as Live shows it (1 for Macro 1). Ignored if macro is also given.
path	string	opt	Path e.g. t1/d1.
track_number	integer	opt	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
track_type	midi | audio | return | master	opt	—
value	number	opt	Value in native units.
value_normalized	number	opt	Value normalized from 0.0 to 1.0.

Note: Addressing by name survives someone reordering the macros; addressing by number does not.

live_map_rack_magnitude

HEAVY LIVE: YES

Scans a Rack for Dry/Wet-style parameters across all its chains and binds them to Macro 1 as a single 'Magnitude' control — one knob for the overall intensity of the whole Rack.

Machine description: *Analyze an Audio or MIDI Effect Rack, discover Dry/Wet or Mix/Amount blend parameters across all nested chains and devices, rename Macro 1 (or specified macro) to 'Magnitude', and set the overall wet/dry magnitude level. `mapped` in the response is always false and `macro_map_api_supported` only reflects whether the underlying macro_map call didn't throw -- Live's API gives no way to verify a macro mapping actually took, so magnitude control instead relies on `set_directly` writing each blend parameter's value directly, which is what reliably works regardless of mapping.*

Argument	Type		What it's for
track_number	integer	req	Target track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
blend_names	array of string	opt	Optional custom parameter names to match as blend controls (defaults to Dry/Wet, Mix, Amount, Blend, Wet).
device_number	integer	opt	Device number of the Rack (default 1).
macro_name	string	opt	Name for the macro (default 'Magnitude').
macro_number	integer	opt	Macro number, exactly as Live shows it (1 for Macro 1, default 1).
value	number	opt	Raw macro value to set immediately.
value_normalized	number	opt	Normalized magnitude value (0.0=dry, 1.0=full wet) to set immediately across the macro and all mapped blend parameters.

Note: PARTIALLY INERT: the mapping works, but the macro-renaming step relies on properties absent from real Racks, so the knob won't be relabelled. See Book 4.

live_read_drum_pads

MEDIUM LIVE: YES

Inspects every active pad in a Drum Rack: which note triggers it, what it's called, whether it's muted or soloed, and what's in its chain.

Machine description: *Inspect all active drum pads in a Drum Rack device with their note numbers, pad names, mute, solo, and chain counts.*

Argument	Type		What it's for
`device_number`	integer	req	Device number of the Drum Rack.
`track_number`	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`all`	boolean	opt	Whether to return all 128 pads or only pads with assigned chains (default false).

Note: A full kit has a lot of pads — this is the second-largest device read after a big plugin.

`live_update_drum_pad`

FEATHER LIVE: YES

Renames, mutes or solos one specific Drum Rack pad.

Machine description: *Update or customize a specific drum pad in a Drum Rack (rename, mute, solo).*

Argument	Type		What it's for
`device_number`	integer	req	Device number of the Drum Rack.
`pad_note`	string	req	Pad note number or name, e.g. 36 or 'C1'.
`track_number`	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`mute`	boolean	opt	Mute state for the pad.
`name`	string	opt	New custom display name for the pad.
`solo`	boolean	opt	Solo state for the pad.

Note: Addressed by the MIDI note the pad responds to, not by position on screen.

`live_inspect_plugin`

MEDIUM LIVE: YES

Deep discovery on any device: is it native, VST, AU or Max for Live, what class is it, what are all its parameters, and what presets does it expose.

Machine description: *Deep device and plugin discovery on a track. Identifies whether a device is native, VST/AU plugin, or Max for Live, and extracts class names, display names, total parameter count, and all exposed VST/AU plugin presets. After inspect, use live_read_device / live_update_device for Live-visible knobs. presets is a catalog, not a load command.*

Argument	Type		What it's for
`device_number`	integer	req	Device number.
`track_number`	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: The tool to reach for when Genius says it can't find a parameter you can clearly see in the GUI.

`live_read_param_range`

FEATHER LIVE: YES

Asks one parameter what values it will actually accept: its minimum and maximum, whether it snaps to steps, its list of named options if it has one, and what it currently displays.

Machine description: *Read a device parameter's min/max, whether it's quantized, its enum value list (if any), and its current display value -- use this to pick a legal value before writing one, instead of guessing.*

Argument	Type		What it's for
`device_number`	integer	req	Device number on that track, exactly as Live shows it.

Argument	Type		What it's for
`track_number`	integer	**req**	Track number, exactly as Live shows it.
`param_name`	string	opt	Exact parameter name (case-insensitive). Give this or param_number.
`param_number`	integer	opt	Parameter number, exactly as live_read_device's params list shows it.

Note: Added in v1.0.39 specifically so Genius stops guessing. Call this before writing an unfamiliar parameter.

`live_set_param_raw`

FEATHER LIVE: YES

Writes a parameter's underlying raw number directly, skipping all display-string formatting.

Machine description: *Set a device parameter's raw underlying value directly, within its min/max, bypassing display-string formatting entirely. The honest escape hatch when a device's display-string formatter is broken or ambiguous.*

Argument	Type		What it's for
`device_number`	integer	**req**	Device number on that track, exactly as Live shows it.
`track_number`	integer	**req**	Track number, exactly as Live shows it.
`value`	number	**req**	Raw value to write, clamped to the parameter's min/max.
`param_name`	string	opt	Exact parameter name (case-insensitive). Give this or param_number.
`param_number`	integer	opt	Parameter number, exactly as live_read_device's params list shows it.

Note: The honest escape hatch for devices whose display formatter is broken or ambiguous. Pair it with live_read_param_range so you write a legal value.

`live_set_appointed_device`

FEATHER LIVE: YES

Intended to appoint a device as the target that hardware control surfaces follow.

Machine description: *Appoint a device as the active target for external control surfaces (hardware controllers bound to 'the appointed device').*

Argument	Type		What it's for
`device_number`	integer	**req**	Device number on that track, exactly as Live shows it.
`track_number`	integer	**req**	Track number, exactly as Live shows it.

Note: UNRELIABLE. The write does not persist in testing; the tool now reports an honest failure rather than pretending. See Book 4.

Algorithmic & Generative MIDI

Deterministic music generation. These compute a pattern in Python and write real notes into Live — there is no model improvisation in the note data itself.

`live_generate_euclidean`

LIGHT LIVE: HYBRID

Spreads a number of hits as evenly as mathematically possible across a number of steps, using the Bjorklund algorithm. This is the engine behind a huge amount of techno, Afro-Cuban and IDM rhythm.

Machine description: *Algorithmic Euclidean rhythm generator (Bjorklund algorithm). Distributes pulses evenly across steps and writes directly to a Session clip.*

Argument	Type		What it's for
`pulses`	integer	req	Number of active hits/pulses (e.g. 5).
`slot_number`	integer	req	Target clip slot number.
`track_number`	integer	req	Target track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`bars`	integer	opt	Number of measures/bars to write (default 1).
`humanize`	number	opt	Timing and velocity jitter (0.0 to 1.0).
`pitch`	integer	opt	MIDI note pitch (e.g. 36 for kick, 38 snare, 42 hat).
`rotate`	integer	opt	Shift pattern forward by N steps.
`steps`	integer	opt	Total steps in cycle (e.g. 16 for 16th notes).
`velocity`	integer	opt	Base velocity 1-127 (default 100).

Note: Fully deterministic: the same inputs always produce the same rhythm. Great for 'give me three variations' by varying pulses.

`live_generate_chords`

LIGHT LIVE: HYBRID

Writes a chord progression with real voice leading — drop-2, rootless and other voicings — rather than stacking root-position triads.

Machine description: *Generate voiced chord progressions with smooth voice leading (Drop-2, rootless A/B, shell voicings) and write directly to a Session clip.*

Argument	Type		What it's for
`progression`	string	req	Progression string e.g. 'ii-V-I', 'Dm7 G7 Cmaj7', or 'I vi IV V'.
`slot_number`	integer	req	Target clip slot number.
`track_number`	integer	req	Target track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`beats_per_chord`	number	opt	Duration of each chord in beats (default 4.0).
`center_pitch`	integer	opt	Center MIDI pitch register (default 60 = C3).
`key`	string	opt	Key root note (e.g. 'C', 'F#', 'Eb', default 'C').
`strum`	number	opt	Strumming delay between notes in beats (e.g. 0.015).
`style`	drop2 | rootless | shell | open | root	opt	Voicing style (default drop2).

Argument	Type		What it's for
`velocity`	integer	opt	Chord velocity (default 85).

Note: The voice-leading logic is what stops it sounding like a beginner's MIDI file.

`live_generate_walking_bass`

LIGHT LIVE: HYBRID

Builds a jazz/blues quarter-note walking bassline over a chord progression, including chromatic approach notes into each change.

Machine description: *Generate a jazz/blues quarter-note walking bassline with root anchors, diatonic steps, and chromatic approaches.*

Argument	Type		What it's for
`progression`	string	req	Progression string e.g. 'ii-V-I-I', 'Dm7-G7-Cmaj7-Cmaj7'.
`slot_number`	integer	req	Target clip slot number.
`track_number`	integer	req	Target track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`beats_per_chord`	number	opt	Beats per chord (default 4.0).
`ghost`	boolean	opt	Include syncopated ghost notes (default true).
`key`	string	opt	Key root (default 'C').
`octave`	integer	opt	Bass octave register (1 = low, 2 = mid).
`velocity`	integer	opt	Base velocity (default 95).

Note: Needs a chord progression as input — it walks between changes, so it has to know what the changes are.

`live_transform_clip`

LIGHT LIVE: HYBRID

Applies a classical compositional transformation to an existing clip: invert it, play it backwards, stretch or compress the rhythm, transpose it, or humanize the timing.

Machine description: *Apply classical motif transformations to a MIDI clip's notes: pitch inversion, retrograde (reverse), retrograde inversion, augmentation (time stretch), diminution (compression), transposition, or humanization.*

Argument	Type		What it's for
`op`	invert | retrograde | retrograde_invert | augment | diminish | transpose | humanize	req	Transformation operation.
`slot_number`	integer	req	Clip slot number.
`track_number`	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`amount`	number	opt	Scaling factor (for augment/diminish) or semitone shift (for transpose).
`axis`	integer	opt	Optional pitch axis for inversion (defaults to median pitch).

Note: Reads the clip, transforms locally, writes it back. Non-destructive only if you write to a different slot.

`live_write_drum_grid`

LIGHT LIVE: HYBRID

Takes an ASCII drum grid — the classic text layout with a row per drum and x's for hits — and turns it into real timed notes on a Drum Rack track.

Machine description: *Parse a multi-line ASCII drum grid into timed MIDI note events and write directly to a Session clip on a Drum Rack track.*

Argument	Type		What it's for
`grid`	string	req	Multi-line ASCII grid e.g. 'K: x...x...x...x...\nS: ....x.....x...\nCH: x.x.x.x.x.x.x.x.'
`slot_number`	integer	req	Target clip slot number.
`track_number`	integer	req	Target drum track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`step_beats`	number	opt	Step length in beats (default 0.25 = 16th note).
`swing`	number	opt	Swing amount 0.0 to 1.0 (default 0.0).

Note: The most legible way to specify a beat in a prompt, because you can literally see the pattern.

`live_create_drum_pattern`

LIGHT LIVE: HYBRID

A step-string drum writer with velocity dynamics and polyrhythm support — more expressive than the plain grid, with accents and cross-rhythms.

Machine description: *Write a drum pattern into a MIDI clip using step strings with velocity dynamics ('2'-'9') and polyrhythms. Pattern is a dict {voice: step_string} where voice can be 'kick', 'snare', 'hat', 'clap', 'tom_low', 'ride', 'shaker', note name ('C1'), or MIDI pitch int. Step chars: 'x'/'1' = hit, '.'/'-'/' ' = rest, '2'-'9' = scaled velocity (2 is ghost ~28, 9 is max ~127). Rows can have different lengths for polyrhythms.*

Argument	Type		What it's for
`pattern`	object	req	Dict of {voice: step_string} e.g. {'kick': 'x...x...x...x...', 'snare': '....x.....x...'}
`slot_number`	integer	req	Target clip slot number.
`track_number`	integer	req	Target track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`beats_per_bar`	number	opt	Beats per measure (default 4.0).
`default_velocity`	integer	opt	Base hit velocity (default 100).
`length_beats`	number	opt	Explicit total clip duration in beats (optional).
`name`	string	opt	Clip name.

Note: Use this over live_write_drum_grid when you care about the dynamics, not just the placement.

`live_create_chord_progression`

LIGHT LIVE: HYBRID

Writes a progression specified either in Roman numerals or by chord names, with control over voicing.

Machine description: *Write a voiced chord progression into a MIDI clip from Roman numeral tokens ('I', 'vi', 'IV', 'V', 'ii°', 'bVII', 'V7') or explicit chord names ('Cm', 'F#maj7', 'G7'). Supports voicings: 'triad', 'seventh', 'power'. beats_per_chord can be a single number or per-chord list.*

Argument	Type		What it's for
`chords`	array of string	req	List of Roman tokens ('I', 'vi', 'IV', 'V', 'ii°', 'bVII') or chord names ('Cm', 'F#maj7'). Also accepts a single space/comma-separated string.
`slot_number`	integer	req	Target clip slot number.
`track_number`	integer	req	Target track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
`base_octave`	integer	opt	Base octave register (default 3 = C3).
`beats_per_chord`	number	opt	Duration per chord in beats (default 4.0).

Argument	Type		What it's for
`key`	string	opt	Key root and mode e.g. 'C', 'Am', 'F#', 'Eb' (default 'C').
`length_beats`	number	opt	Explicit total clip length in beats (optional).
`name`	string	opt	Clip name.
`velocity`	integer	opt	Chord velocity (default 85).
`voicing`	triad | seventh | power	opt	Voicing style (default 'triad').

Note: Roman numerals let you say the progression once and try it in any key.

`live_detect_key`

LIGHT LIVE: HYBRID

Works out what key and mode a clip is in, using the Krumhansl-Schmuckler algorithm — a published, deterministic method, not a guess from the language model.

Machine description: *Deterministic key/mode detection (Krumhansl-Schmuckler pitch-class analysis) for a MIDI clip, track(s), or the whole Set. Always call this for 'what key/scale is this in' questions instead of guessing from live_read_clip note data yourself — the math is exact, eyeballing notes is not.*

Argument	Type		What it's for
`profile`	krumhansl-kessler | temperley	opt	Weight table to correlate against. Default krumhansl-kessler.
`results`	integer	opt	How many top key/mode guesses to return (1-12, default 3).
`scope`	clip | track | set	opt	clip = selected/detail MIDI clip (default). track = all clips on given track_number(s). set = every MIDI clip in the Set.
`track_numbers`	array of integer	opt	Track number(s) for scope=track, exactly as Live shows them, e.g. [1] or [1,3].

Note: Because it's deterministic it will give the same answer every time, and it will tell you honestly when the material is ambiguous.

MIDI Editor Note Selection

Added in v1.0.39. Operates on the notes you have physically selected in Live's piano roll — the closest thing to Genius reaching over and using your mouse.

`live_read_selected_notes`

LIGHT LIVE: YES

Reads exactly the notes you currently have highlighted in the MIDI editor, so you can say 'these ones' instead of describing a time range.

Machine description: *Read only the MIDI notes currently selected in the piano-roll/MIDI Note editor for a clip -- unlike every other note tool, this does not touch or return the whole clip.*

Argument	Type		What it's for
`slot_number`	integer	req	Clip slot number, exactly as Live shows it.
`track_number`	integer	req	Track number, exactly as Live shows it.

Note: Correctly refuses if the clip is audio rather than MIDI. Returns an empty list, not an error, when nothing is selected.

`live_transform_selected_notes`

LIGHT LIVE: HYBRID

Applies a transformation to just the notes you've selected in the piano roll, leaving the rest of the clip alone.

Machine description: *Apply a classical motif transformation (invert, retrograde, transpose, humanize, etc.) to only the notes currently selected in the MIDI editor, leaving the rest of the clip untouched. The user must select notes in Live's piano roll first; use live_transform_clip instead to transform an entire clip.*

Argument	Type		What it's for
`op`	invert | retrograde | retrograde_invert | augment | diminish | transpose | humanize	req	—
`slot_number`	integer	req	Clip slot number, exactly as Live shows it.
`track_number`	integer	req	Track number, exactly as Live shows it.
`amount`	number	opt	Scaling factor (augment/diminish) or semitone shift (transpose).
`axis`	integer	opt	Optional pitch axis for inversion (defaults to the median pitch of the selection).

Note: Still the least battle-tested action in the set — as of v1.0.40 it had not been exercised with a real non-empty selection. See Book 4.

Genre Recipes

Pre-built multi-step production templates that run as a single call.

`live_list_recipes`

FEATHER LIVE: NO

Shows the catalogue of built-in multi-step recipes available to run.

Machine description: *Read-only catalog of embedded multi-step music production recipes (tech-house kicks, reese bass, 808 subs, lofi jazz/neo-soul chord voicings, parallel comp racks, master bus/vocal chains, arrangements). Returns recipe names, categories, descriptions, and input parameters. `available:false` means the recipe needs a capability (e.g. the Remote Script companion for real browser presets) this build doesn't have yet -- don't call live_apply_recipe on it.*

Argument	Type		What it's for
`category`	drums | bass | chords | racks | arrangements | mixing | live_performance	opt	Optional category filter.

Note: Runs entirely locally — you can browse the catalogue without Ableton even open.

`live_apply_recipe`

HEAVY LIVE: HYBRID

Executes a whole recipe by ID: creates the tracks, loads the devices, sets the parameters and writes the MIDI, all in one call.

Machine description: *Execute an embedded genre recipe by ID, automatically building multi-step production sequences (creating tracks, loading devices, tweaking parameters, and writing MIDI clips/notes) in Ableton Live.*

Argument	Type		What it's for
`recipe_id`	string	req	The recipe identifier, e.g. 'drums/tech-house-kick', 'bass/reese', 'racks/parallel-comp'.
`overrides`	object	opt	Optional parameter overrides declared by the recipe (see live_list_recipes' inputs for that recipe). A recipe with a track_number input takes it 1-based, exactly as Live shows it -- never a raw track_index override, which is rejected.

Note: The single highest-leverage action in the toolset, and the most disruptive. Take a live_session_snapshot first.

Audio Analysis, Recording & Mix

Genius's ears. These are the only actions that judge how your music actually sounds rather than what it's made of.

`live_analyze_audio`

MEDIUM LIVE: NO

Measures an audio file properly: LUFS loudness, true peak, RMS, crest factor, a 10-band spectrum, plus brightness and stereo width.

Machine description: *Objective acoustic analysis of a WAV audio file: Integrated/Short-term LUFS, True Peak (dBTP), RMS, crest factor, 10-band spectral balance, spectral centroid (brightness Hz), rolloff, flatness, transient punch (band crest dB), and stereo width/correlation.*

Argument	Type		What it's for
`path`	string	req	Absolute or relative path to the audio file.
`max_seconds`	number	opt	Maximum seconds to analyze (default 600).

Note: Pure local DSP on a file on disk — Ableton doesn't need to be running. Real numbers, not an opinion.

`live_compare_audio`

MEDIUM LIVE: NO

Diffs two audio files and reports what measurably changed between them — the before and after of a mix decision.

Machine description: *Diff and compare two audio files (B relative to A) to evaluate perceptual and acoustic changes before and after a parameter or processing tweak. Returns headline and delta metrics (LUFS, punch, brightness, stereo width).*

Argument	Type		What it's for
`path_a`	string	req	Before audio file path.
`path_b`	string	req	After audio file path.
`max_seconds`	number	opt	Maximum seconds to analyze (default 600).

Note: The objective way to answer 'did that actually help?'

`live_record_audio`

VERY HEAVY LIVE: YES

Captures whatever Live is currently outputting to a WAV file, via Windows loopback recording.

Machine description: *Capture Ableton Live's real-time audio output to a WAV file via a Windows loopback audio device (e.g. Voicemeeter VAIO / virtual cable) using ffmpeg.*

Argument	Type		What it's for
`out_path`	string	opt	Optional output WAV path.
`scene_number`	integer	opt	Optional scene number to fire at start.
`seconds`	number	opt	Duration to record in seconds (default 4.0).
`start_playback`	boolean	opt	Whether to start Live transport playback (default true if recording).
`track_number`	integer	opt	Optional track number to solo during capture, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: Real time: a 30-second capture takes 30 seconds. This is how audio gets from Live into the analysis tools at all.

`live_record_and_analyze`

VERY HEAVY LIVE: YES

Records Live's output and immediately measures it, in one call — closing the loop so Genius can tune a parameter, listen, and tune again.

Machine description: *Capture real-time audio output from Live and immediately run acoustic DSP analysis in one call. The core primitive for closed-loop parameter tuning.*

Argument	Type		What it's for
`scene_number`	integer	opt	Optional scene number to fire at start.
`seconds`	number	opt	Duration to record in seconds (default 4.0).
`start_playback`	boolean	opt	Whether to start Live transport playback.
`track_number`	integer	opt	Optional track number to solo during capture, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).

Note: Real time plus analysis. The foundation of any 'keep adjusting until it hits -14 LUFS' workflow.

`live_analyze_mix`

HEAVY LIVE: HYBRID

Audits the Set structurally and flags the usual suspects: low-end clashes between tracks, clipping, a missing limiter, lopsided panning.

Machine description: *Perform a heuristic structural mix audit on the running Live Set: flags low-end clashing frequencies, clipping tracks, missing master limiters, solo hygiene, and panning balance.*

Arguments: none — just call it.

Note: Heuristic and structural — it reasons about your Set's layout, it does not listen. Combine with live_record_and_analyze for the full picture.

Samples & Browser

Finding sounds on disk and in Live's own library.

`live_library`

MEDIUM LIVE: HYBRID

Searches your configured Samples folder on disk, including tags and categories, and can also reach into Live's browser.

Machine description: *Search the user's disk Samples folder (Live's own browser cannot be searched through the Live Object Model, so search/searchBatch only ever return disk matches). listTags/listCategories return a static category list; listPlugins cannot see installed plug-ins.*

Argument	Type		What it's for
`action`	search | searchBatch | listTags | listCategories | listPlugins	opt	—
`category`	string	opt	—
`kind`	audio | midi | preset | device-group | m4l-device | plugin | folder	opt	—
`limit`	number	opt	—
`queries`	array of object	opt	—
`query`	string	opt	—

Note: The disk search half runs locally and is fast; a large uncategorised sample folder is the main thing that slows it down.

`live_find_samples`

LIGHT LIVE: NO

A quick free-text hunt for sample files by name.

Machine description: *Search the user's Samples folder on disk for one-shots (kicks, snares, 808s, hats).*

Argument	Type		What it's for
`query`	string	req	—
`category`	all | kick | snare | clap | hat | tom | perc | 808	opt	—

Note: Entirely local. The fastest way to turn 'a punchy kick' into an actual file path you can load.

Knowledge Base

Where Genius's music knowledge comes from. Two separate sources, deliberately, with a strict order of preference.

`library_db_search`

LIGHT LIVE: NO

Searches the structured Music/Audio database — 42 curated datasets across 9 categories covering theory, production, sound design, Ableton technique and the Live Object Model. Always the first place Genius looks.

Machine description: *Full-text search (SQLite FTS5) over the Music/Audio knowledge database — the preferred first stop for Ableton, music theory, sound design, production/DJ, Novation/Launchkey, drum pads, M4L devices, audio-DSP, live-coding (Sonic Pi), generative VSTs, and third-party plugin/hardware manual questions. Only the official Live PDF and a few remaining manuals still live in TRAINING DOCS (use library_read/library_search for those). 40+ topics total, including a family of ~15 Ableton community-tool datasets (AbletonOSC, PyLive, AbletonMCP, AbletonBridge, LiveRPyC, the Live 12 Python API/LOM references, etc.) and ~8 third-party device-manual datasets (Native Instruments, Ozone, Surge, Oberheim, Klevgrand, etc.) — ALWAYS call with topic='?' first if you're not sure the exact key, rather than guessing or assuming a topic doesn't exist.*

Argument	Type		What it's for
`query`	string	req	—
`topic`	string	req	A topic key, e.g. ableton_training, sound_design, music_theory, production_djing, drum_pads, novation, m4l_device_info, m4l_general, audio_analysis, audio_splitting, misc, new_research, research_results, liveapi, liveapi_lom, music_production, surge, sonic_pi, infinite_crate, ableton_god_producer, mixology_db, cycling74_max_reference, hw_native_instruments (see topic='?' for the full current list, including the rest of the Ableton community-tool and hardware-manual families).
`limit`	integer	opt	1-10, default 5.

Note: A fast indexed full-text search. You must name which topic to search, which is why the INDEX.md router exists.

`library_search`

MEDIUM LIVE: NO

Full-text search across the raw Training Docs folder — the official Ableton Live manual, roughly 140 third-party plugin manuals, and assorted exports.

Machine description: *Search the Genius library (TRAINING DOCS + plugin docs) by filename or INDEX row. Use when INDEX has no obvious row.*

Argument	Type		What it's for
`query`	string	req	—
`limit`	integer	opt	—

Note: The fallback, used only when the database genuinely has nothing. Slower because it reads real files rather than an index.

`library_read`

MEDIUM LIVE: NO

Opens one specific document from the Training Docs folder and reads it, once you already know which file you want.

Machine description: *Open one studio-library file (INDEX.md, chapter markdown, or PDF). Always follow Decide first: read INDEX.md, then only the 1–3 books it names.*

Argument	Type		What it's for
`path`	string	req	Absolute path or path relative to the Knowledgebase Library.
`max_chars`	integer	opt	—
`query`	string	opt	Optional substring to jump to (chapter, device, topic).

Note: Long documents get truncated, so ask for the section you need rather than the whole manual.

Offline Project Files (.als)

Reads Ableton project files straight off disk. Live does not need to be running, and the project does not need to be open.

als_summary

LIGHT LIVE: NO

Summarises a saved .als project file without opening it: tempo, tracks, devices, general shape.

Machine description: *Inspect an Ableton Live set (.als) file offline without running Ableton: creator version, tempo, time signature, scale, track count, clips, and locators.*

Argument	Type		What it's for
path	string	req	Path to the .als file.

Note: Genuinely useful for triaging a folder full of old projects you can't remember.

als_list_tracks

LIGHT LIVE: NO

Lists the tracks inside a saved project file.

Machine description: *List all tracks, mixer volumes, pans, loaded devices, and clips directly from a saved .als file offline.*

Argument	Type		What it's for
path	string	req	Path to the .als file.

Note: A quick look inside before deciding whether a project is worth reopening.

als_extract_midi

MEDIUM LIVE: NO

Pulls the MIDI out of a specific track in a saved project file.

Machine description: *Extract MIDI notes from a clip in a saved .als project directly into JSON note events without opening Ableton.*

Argument	Type		What it's for
path	string	req	Path to the .als file.
track_number	integer	req	Track number, exactly as Live shows it (Live's track 1 = track_number 1; digits in a track NAME do not indicate position).
clip_number	integer	opt	MIDI clip number on that track, exactly as Live shows it.

Note: Lets you recover a good part from an old project without loading the whole thing and its missing plugins.

als_diff

MEDIUM LIVE: NO

Compares two saved project files and reports what changed between them.

Machine description: *Diff two saved .als project files (e.g. song_v1.als vs song_v2.als) offline, reporting added/removed tracks, renamed tracks, tempo changes, and clip count differences.*

Argument	Type		What it's for
path_a	string	req	First .als file path.
path_b	string	req	Second .als file path to compare against.

Note: Effectively version control for Ableton projects — 'what did I actually change in v3?'

als_find_unfinished

LIGHT LIVE: NO

Scans a project and flags the signs of unfinished work: empty tracks, default names, missing arrangement.

Machine description: *Lint a saved .als file offline for production omissions, unfinished work, clipping master faders, missing limiters, soloed tracks, and empty clips.*

Argument	Type		What it's for
`path`	string	req	Path to the .als file.

Note: Good for deciding which of forty half-ideas is closest to being finishable.

Diagnostics, Views & Escape Hatches

The tools for when something isn't working, plus direct access to Live's own API for anything the purpose-built actions don't cover.

`live_context`

MEDIUM LIVE: HYBRID

Gathers a broad picture of the current working context for Genius to reason over.

Machine description: *Persistent production memory. scope=project (this set), global (who the user is), memory (named facts). Prefer read before write.*

Argument	Type		What it's for
`action`	read | write | delete	opt	—
`content`	string	opt	—
`description`	string	opt	One-line recall hook for a memory write.
`name`	string	opt	Memory entry name.
`project`	string	opt	Live Set name for project scope.
`scope`	project | global | memory	opt	—

Note: Usually called automatically at the start of a session rather than by you directly.

`live_api`

LIGHT LIVE: YES

Raw access to the Live Object Model: get, set and call any property or method Live exposes, including multi-argument calls.

Machine description: *Raw Live Object Model access (get/set/call/goto). Use only when a specialized live_* tool cannot do the job.*

Argument	Type		What it's for
`operations`	array of object	req	—
`path`	string	opt	e.g. live_set tracks 0.

Note: The ultimate escape hatch and the sharpest tool here — it will happily do things no purpose-built action would let you. Extended in v1.0.40 to support multi-argument sets.

`live_run_action`

MEDIUM LIVE: HYBRID

Runs one of the app's own bundled tool scripts by ID — the helpers that ship in the tools folder.

Machine description: *Run a built-in Live Actions host tool. payload is {"tracks": [...], "fields": {...}}. `tracks` is a list of {"track_number": N, "kind": "midi"|"audio"|"return"|"master"} for the tracks this action should run on, track_number exactly as Live shows it -- call live_read_set first and build this list from the tracks the user is talking about (by name or by the track number they said); most tool_ids require at least one track and return an error message (not a hard failure) like 'Tick a MIDI track.' if the list is empty. `fields` holds tool-specific options, all optional unless noted: clip-namer {pattern: "{track} {bar} take"}; color-clips {color: "#3b82f6" (required), also_scenes: "yes"|"no"}; empty-clips {length: 4, name: "Empty"} (MIDI tracks only); mutate {count: 4, seed: 1, mode: "independent"} (needs a MIDI clip selected on a ticked MIDI track); track-batch {action: "rename", pattern: "{name}"}; duplicate-clips {} (copies each ticked track's clips into its own empty slots); reveal-device {name: "Chord Explorer"}; arrangement-skeleton, breakdowns, bbencut, arrangement-helper: see live_context for their fields. detect-key: prefer the dedicated live_detect_key tool instead of this one. Async tool_ids (arrangement-helper section placement, bbencut) return {ok: true, pending: true, message: "..."} immediately and keep working in the background; that is a normal 'started' response, not a failure.*

Argument	Type		What it's for
`tool_id`	clip-namer | arrangement-skeleton | mutate | empty-clips | color-clips | duplicate-clips | track-batch | reveal-device | breakdowns | bbencut | arrangement-helper	req	—
`payload`	object	opt	—

Note: Cost depends entirely on which tool you run; some are trivial and some rebuild whole sections of a Set.

`live_focus_view`

FEATHER LIVE: YES

Brings a particular part of Live's interface into focus.

Machine description: *Focus a primary Live interface view (Session, Arrangement, Detail, Clip, Device) and optionally toggle Draw or Follow modes.*

Argument	Type		What it's for
`draw_mode`	boolean	opt	Toggle pencil draw mode on/off.
`follow_mode`	boolean	opt	Toggle arrangement playhead follow mode on/off.
`view`	session | arrangement | detail | clip | device	opt	View window to focus.

Note: Cosmetic, but it means you're looking at the right thing when Genius describes what it did.

`live_view_control`

FEATHER LIVE: YES

Lists which of Live's main views exist, and checks or changes whether one is currently visible.

Machine description: *Show, hide, focus, or query visibility of a Live UI view (Session, Arranger, Detail/Clip, Detail/DeviceChain, Browser, etc.). action='list' (default) returns the valid view names for this Live version.*

Argument	Type		What it's for
`action`	list | show | hide | focus | is_visible	opt	Default 'list'.
`view_name`	string	opt	Required for show/hide/focus/is_visible -- one of the names action='list' returns.

Note: The list mode was returning only one entry until v1.0.40 fixed an unwrapping bug; it now correctly returns all six views.

`live_show_message`

FEATHER LIVE: YES

Posts a short message into Live's own status bar at the bottom of the window.

Machine description: *Show a short message in Live's own status bar (bottom-left of the Live window) -- a feedback channel visible to the user that isn't Max Console.*

Argument	Type		What it's for
`text`	string	req	—

Note: A feedback channel you can actually see, unlike Max Console. Good for long batch jobs announcing progress.

`live_read_dialog`

FEATHER LIVE: YES

Detects a modal dialog currently blocking Live's interface, reads what it says, and can press one of its buttons.

Machine description: *Read the message/button count of a Live modal dialog currently blocking the UI, or press one of its buttons to dismiss it. Use this when another live_* call unexpectedly times out -- a stuck modal is a likely cause.*

Argument	Type		What it's for
`press_button_number`	integer	opt	Button number to press (1-based). Omit to just read the current dialog message.

Note: Reach for this first when a live_* call mysteriously times out — a stuck modal is the most common cause.

`live_read_cpu`

FEATHER LIVE: YES

Reads Live's own average and peak audio-engine CPU usage.

Machine description: *Read Live's own average/peak audio engine CPU usage.*

Arguments: none — just call it.

Note: Lets Genius notice it has loaded you into crackling territory before you do.

Alphabetical index

All 122 actions with their family and weight.

Action	Family	Weight
als_diff	Offline Project Files (.als)	Medium
als_extract_midi	Offline Project Files (.als)	Medium
als_find_unfinished	Offline Project Files (.als)	Light
als_list_tracks	Offline Project Files (.als)	Light
als_summary	Offline Project Files (.als)	Light
library_db_search	Knowledge Base	Light
library_read	Knowledge Base	Medium
library_search	Knowledge Base	Medium
live_add_shaped_automation	Clip Automation & Envelopes	Medium
live_analyze_audio	Audio Analysis, Recording & Mix	Medium
live_analyze_mix	Audio Analysis, Recording & Mix	Heavy
live_api	Diagnostics, Views & Escape Hatches	Light
live_apply_recipe	Genre Recipes	Heavy
live_back_to_arranger	Transport, Tempo & Global Feel	Feather
live_batch_tracks	Transport, Tempo & Global Feel	Light
live_capture_midi	Arrangement Timeline	Light
live_clear_clip_envelopes	Clip Automation & Envelopes	Light
live_commit_groove	Session Clips — Create & Edit	Light
live_compare_audio	Audio Analysis, Recording & Mix	Medium
live_connect	Connect & Read Live State	Feather
live_context	Diagnostics, Views & Escape Hatches	Medium
live_continue_playing	Transport, Tempo & Global Feel	Feather
live_create_chord_progression	Algorithmic & Generative MIDI	Light
live_create_clip	Session Clips — Create & Edit	Light
live_create_device	Devices, Racks & Macros	Light
live_create_drum_pattern	Algorithmic & Generative MIDI	Light
live_create_scene	Scenes	Light
live_create_take_lane	Take Lanes & Comping	Feather
live_create_take_lane_clip	Take Lanes & Comping	Light
live_create_track	Tracks & Mixer	Light
live_delete	Tracks & Mixer	Feather
live_detect_key	Algorithmic & Generative MIDI	Light

Action	Family	Weight
live_duplicate	Tracks & Mixer	Light
live_duplicate_loop	Session Clips — Create & Edit	Light
live_duplicate_to_arrangement	Arrangement Timeline	Light
live_edit_clip	Session Clips — Create & Edit	Light
live_edit_scene	Scenes	Light
live_edit_track	Tracks & Mixer	Light
live_find_samples	Samples & Browser	Light
live_focus_view	Diagnostics, Views & Escape Hatches	Feather
live_generate_chords	Algorithmic & Generative MIDI	Light
live_generate_euclidean	Algorithmic & Generative MIDI	Light
live_generate_walking_bass	Algorithmic & Generative MIDI	Light
live_get_clip_automation	Clip Automation & Envelopes	Feather
live_get_clip_follow_action	Session Clips — Create & Edit	Feather
live_get_selected	Connect & Read Live State	Feather
live_groove_pool	Session Clips — Create & Edit	Light
live_inspect_plugin	Devices, Racks & Macros	Medium
live_jump_cue	Transport, Tempo & Global Feel	Feather
live_jump_to_time	Transport, Tempo & Global Feel	Feather
live_library	Samples & Browser	Medium
live_list_recipes	Genre Recipes	Feather
live_manage_warp_markers	Session Clips — Create & Edit	Light
live_map_rack_magnitude	Devices, Racks & Macros	Heavy
live_move_arrangement_clip	Arrangement Timeline	Light
live_move_device	Devices, Racks & Macros	Feather
live_nudge	Transport, Tempo & Global Feel	Feather
live_playback	Transport, Tempo & Global Feel	Feather
live_re_enable_automation	Transport, Tempo & Global Feel	Feather
live_read_arrangement_clips	Arrangement Timeline	Light
live_read_arrangement_length	Arrangement Timeline	Feather
live_read_arrangement_notes	Arrangement Timeline	Medium
live_read_clip	Session Clips — Create & Edit	Medium
live_read_cpu	Diagnostics, Views & Escape Hatches	Feather
live_read_device	Devices, Racks & Macros	Medium
live_read_dialog	Diagnostics, Views & Escape Hatches	Feather
live_read_drum_pads	Devices, Racks & Macros	Medium

Action	Family	Weight
live_read_link_status	Connect & Read Live State	Feather
live_read_macros	Devices, Racks & Macros	Light
live_read_meter	Connect & Read Live State	Feather
live_read_param_range	Devices, Racks & Macros	Feather
live_read_scene	Scenes	Light
live_read_selected_notes	MIDI Editor Note Selection	Light
live_read_set	Connect & Read Live State	Medium
live_read_take_lanes	Take Lanes & Comping	Feather
live_read_track	Tracks & Mixer	Medium
live_record_and_analyze	Audio Analysis, Recording & Mix	Very heavy
live_record_audio	Audio Analysis, Recording & Mix	Very heavy
live_record_to_arrangement	Arrangement Timeline	Very heavy
live_redo	Transport, Tempo & Global Feel	Feather
live_remove_arrangement_notes	Arrangement Timeline	Light
live_remove_notes	Session Clips — Create & Edit	Light
live_run_action	Diagnostics, Views & Escape Hatches	Medium
live_scrub	Transport, Tempo & Global Feel	Feather
live_select	Tracks & Mixer	Feather
live_session_diff	Connect & Read Live State	Medium
live_session_snapshot	Connect & Read Live State	Heavy
live_set_appointed_device	Devices, Racks & Macros	Feather
live_set_automation_record	Transport, Tempo & Global Feel	Feather
live_set_clip_automation	Clip Automation & Envelopes	Medium
live_set_clip_fades	Session Clips — Create & Edit	Feather
live_set_clip_follow_action	Session Clips — Create & Edit	Feather
live_set_clip_launch_mode	Session Clips — Create & Edit	Feather
live_set_clip_markers	Session Clips — Create & Edit	Feather
live_set_crossfader	Transport, Tempo & Global Feel	Feather
live_set_cue_volume	Transport, Tempo & Global Feel	Feather
live_set_fold_state	Tracks & Mixer	Feather
live_set_groove_amount	Transport, Tempo & Global Feel	Feather
live_set_loop_region	Transport, Tempo & Global Feel	Feather
live_set_overdub	Transport, Tempo & Global Feel	Feather
live_set_param_raw	Devices, Racks & Macros	Feather
live_set_punch	Transport, Tempo & Global Feel	Feather

Action	Family	Weight
live_set_swing	Transport, Tempo & Global Feel	Feather
live_show_message	Diagnostics, Views & Escape Hatches	Feather
live_stop_all_clips	Transport, Tempo & Global Feel	Feather
live_tap_tempo	Transport, Tempo & Global Feel	Feather
live_transform_clip	Algorithmic & Generative MIDI	Light
live_transform_selected_notes	MIDI Editor Note Selection	Light
live_undo	Transport, Tempo & Global Feel	Feather
live_update_arrangement_clip	Arrangement Timeline	Feather
live_update_clip	Session Clips — Create & Edit	Light
live_update_device	Devices, Racks & Macros	Light
live_update_drum_pad	Devices, Racks & Macros	Feather
live_update_macro	Devices, Racks & Macros	Feather
live_update_scene	Scenes	Feather
live_update_set	Transport, Tempo & Global Feel	Light
live_update_take_lane	Take Lanes & Comping	Feather
live_update_track	Tracks & Mixer	Light
live_view_control	Diagnostics, Views & Escape Hatches	Feather
live_write_arrangement_notes	Arrangement Timeline	Medium
live_write_drum_grid	Algorithmic & Generative MIDI	Light
live_write_notes	Session Clips — Create & Edit	Medium